THE UNIVERSITY OF ZAMBIA

SCHOOL OF NATURAL AND APPLIED SCIENCES

DEPARTMENT OF COMPUTING AND INFORMATICS

FINAL YEAR PROJECT REPORT

ACADEMIC YEAR 2026

Student Information Management System Design and Implementation with Moodle Integration

A Final Year Project Report Submitted to the

Department of Computing and Informatics

in partial fulfilment of the requirements for the award of

Bachelor of Science in Computer Science

By

Chitundu Milimbo and Charles Hangoma

2020992574 and 2021414469

Supervisor: Professor Jackson Phiri

Date of Submission: 16 October 2026

DECLARATION

We, Chitundu Milimbo and Charles Hangoma, declare that this report describes our shared project work. We acknowledge external sources, collaboration and software assistance. Our account distinguishes implemented functions, verified results and work that remains incomplete.

We will each verify our individual contribution statement before signing. This joint project account does not assign exclusive authorship of shared work to either partner.

Student names: Chitundu Milimbo and Charles Hangoma

Computer numbers: 2020992574 and 2021414469

Chitundu Milimbo signature: _____

Charles Hangoma signature: _____

Date: 16 October 2026

APPROVAL

We submit this report to Professor Jackson Phiri for examination and approval. The signature below records the supervisor’s decision when completed.

Supervisor: Professor Jackson Phiri

Signature: _____

Date: 16 October 2026

Departmental or examiner approval: _____

ABSTRACT

We designed and implemented a Student Information System integrated with Moodle to coordinate university administration and learning access. The project addresses inconsistent records, fragmented workflows and unclear responsibility when admissions, registration, assessment and learning-platform operations are handled separately. Our aim was to provide an authoritative administrative core and a foundation for governed student support.

We followed a handbook-led iterative development method. We implemented a Next.js web interface, a modular NestJS API and a PostgreSQL database through Prisma. Role and scope checks, versioned decisions, idempotent commands and recorded integration intent connect user actions to durable outcomes. We prepared generated participants and representative university configuration for repeatable evaluation.

Our implementation demonstrates core applicant, registration, finance, assessment and timetable workflows. The local review dataset contains nine formal registrations and thirty-six course registrations. Four participant logins passed connected browser checks. A separate MoodleCloud demonstration contains twelve visible courses and forty-eight memberships across four generated accounts, and learner access was confirmed. Recent student-interface and authentication changes also have focused local verification.

The results support the feasibility of retaining authoritative SIS decisions while separating external learning delivery. They do not establish full bidirectional synchronisation, LTI tool launch, an evaluated AI assistant or institutional production readiness. We conclude that the project provides a functioning administrative foundation and a bounded Moodle demonstration, with live delivery, advanced support and complete operational evaluation remaining necessary to fulfil the wider proposal.

Keywords: student information system; Moodle; registration; interoperability; academic administration.

ACKNOWLEDGEMENTS

We thank Professor Jackson Phiri for supervising our final year project. We acknowledge the University of Zambia and the department for the academic setting of our work. We also acknowledge the open-source communities and primary technical documentation on which our implementation relies. We recognise our collaboration as partners and will verify individual contributions against the project records before signing.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION....13
1.1 Background and Context....13
1.2 Problem Statement....13
1.3 Project Aim....13
1.4 Project Objectives....14
1.5 Research Questions (where applicable)....14
1.6 Scope of the Project....14
1.7 Significance of the Project....14
1.8 Limitations....15
1.9 Organisation of the Report....15
CHAPTER 2: LITERATURE REVIEW....16
2.1 Introduction....16
2.2 Key Concepts and Theories....16
2.3 Existing Systems / Related Work....16
2.4 Comparative Analysis....17
2.5 Research / Knowledge Gap....17
2.6 Conceptual or Theoretical Framework (where applicable)....17
2.7 Chapter Summary....18
CHAPTER 3: METHODOLOGY....19
3.1 Introduction....19
3.2 Research / Development Approach....19
3.3 Requirements / Data Collection....19
3.4 System Architecture / Research Workflow....19
3.5 Tools and Technologies....20
3.6 Ethical and Legal Considerations....20
3.7 Evaluation Method....20
3.8 Chapter Summary....20
CHAPTER 4: SYSTEM ANALYSIS AND DESIGN....21
4.1 Introduction....21
4.2 Stakeholder and User Analysis....21
4.3 Functional Requirements....21
4.4 Non-Functional Requirements....22

4.5 Use Cases / User Stories....22
4.6 System / Solution Architecture....23
4.7 Database / Data Design (where applicable)....25
4.8 Interface / Interaction Design (where applicable)....26
4.9 Algorithms / Models / Technical Design....28
4.10 Chapter Summary....30
CHAPTER 5: IMPLEMENTATION / DEVELOPMENT....31
5.1 Introduction....31
5.2 Development Environment....31
5.3 System Components / Modules....31
5.4 Key Implementation Details....31
5.5 Integration....32
5.6 Challenges and Solutions....32
5.7 Individual Contribution (group projects)....32
5.8 Chapter Summary....33
CHAPTER 6: TESTING, RESULTS AND EVALUATION....34
6.1 Introduction....34
6.2 Testing Strategy....34
6.3 Test Cases and Results....34
6.4 Experimental Results / Performance Results (where applicable)....35
6.5 User Evaluation (where applicable)....35
6.6 Analysis of Results....35
6.7 Objective-by-Objective Evaluation....36
6.8 Individual Results (group projects)....36
6.9 Chapter Summary....36
CHAPTER 7: DISCUSSION....37
7.1 Introduction....37
7.2 Interpretation of Findings....37
7.3 Comparison with Existing Work....37
7.4 Contributions of the Project....37
7.5 Limitations and Implications....38
7.6 Chapter Summary....38
CHAPTER 8: CONCLUSION AND RECOMMENDATIONS....39
8.1 Introduction....39

8.2 Summary of the Project.....39
8.3 Conclusions.....39
8.4 Achievement of Objectives.....39
8.5 Recommendations.....40
8.6 Future Work.....40
REFERENCES.....41
APPENDICES.....42
APPENDIX A: INDIVIDUAL CONTRIBUTION STATEMENT.....42
APPENDIX B: USER MANUAL / INSTALLATION GUIDE.....42
APPENDIX C: ADDITIONAL SYSTEM DESIGNS / DIAGRAMS.....43
APPENDIX D: ADDITIONAL TEST CASES AND RESULTS.....43
APPENDIX E: DATA COLLECTION INSTRUMENTS.....43
APPENDIX F: ADDITIONAL CODE / CONFIGURATION.....44
APPENDIX G: OTHER SUPPORTING MATERIAL.....44

LIST OF FIGURES

Figure 4.1 System context and component boundaries.....24
Figure 4.2 Registration and deferred Moodle delivery sequence.....25
Figure 4.3 Selected domain class relationships.....26
Figure 4.4 Connected student home with simulator enrolment handoff.....27
Figure 4.5 Student timetable view with controlled demonstration responses.....28
Figure 4.6 Timetable publication activity.....29
Figure 4.7 Selected design class boundaries.....30

LIST OF TABLES

Table 2.1 Integration approach comparison.....17
Table 4.1 Principal actors and authority.....21
Table 4.2 Representative requirements.....22
Table 6.1 Verification evidence by checkpoint.....34
Table 6.2 Original objective evaluation.....36

LIST OF ABBREVIATIONS AND ACRONYMS

API: Application Programming Interface

CI: Continuous Integration

DBMS: Database Management System

LMS: Learning Management System

LLM: Large Language Model

LTI: Learning Tools Interoperability

RBAC: Role-Based Access Control

REST: Representational State Transfer

SIS: Student Information System

UI: User Interface

UML: Unified Modeling Language

WCAG: Web Content Accessibility Guidelines

GROUP PROJECT AND CONTRIBUTION RECORD

We prepared this shared account as Chitundu Milimbo and Charles Hangoma. Appendix A records our planned areas of responsibility and the procedure for exchanging implementation and review work. Each partner must produce any separately required individual submission and personally confirm authorship before signing.

CHAPTER 1: INTRODUCTION

1.1 Background and Context

Our project addresses the connection between university administration and everyday learning. A Student Information System maintains applications, registration, academic records and administrative decisions. Moodle provides the learning environment in which lecturers organise course activities and students access learning materials. We designed the SIS to retain authority over formal registration and official academic decisions while connecting learning delivery through explicit integration contracts.

The University of Zambia provides the educational context of our project. We used its institutional setting to shape terminology, programme organisation and representative workflows. Our demonstration configuration is a project configuration, rather than a claim that the University has adopted our software or approved every illustrated rule. We designed configurable structures so that another university can define its own schools, programmes, periods and approval responsibilities.

1.2 Problem Statement

The problem we address is the loss of consistency and accountability when administrative decisions and learning access are handled independently. A student may have a valid registration while an external learning platform remains unavailable. A lecturer may hold provisional marks that have not passed the checks needed for official results. A timetable may contain valid individual sessions but still create room, lecturer or student clashes.

We therefore needed more than pages for entering records. We needed connected workflows that explain their current state, preserve the responsible decision and recover from failure. We also needed a student interface that makes the next action clear without exposing staff-only functions. We investigated these problems through the project handbook, approved design evidence, source implementation and repeatable demonstrations, rather than presenting an unrecorded institutional survey as evidence.

1.3 Project Aim

Our overall aim was to design and implement a modular Student Information System with Moodle integration that coordinates authoritative university workflows and provides a foundation for governed student support. The proposal also envisaged bidirectional learning data exchange, LTI 1.3 tools and an LLM layer. We retain those ambitions and evaluate their actual completion separately from the demonstrated core.

1.4 Project Objectives

Objective 1 was to develop core student records, course management, enrolment and grade workflows, with Moodle provisioning through web services and tools embedded through LTI 1.3. Objective 2 was to introduce governed AI assistance for administrative enquiries, staff summaries, cross-platform risk insights and opt-in wellbeing support.

For evaluation, we decomposed these objectives into observable outcomes: a valid student can complete a formal registration; staff can manage academic workflows under role and scope controls; timetable publication produces a student-specific view; Moodle accounts and courses can be provisioned; and any unsupported external action returns an explicit failure rather than a false success. Table 6.2 evaluates the original objectives without redefining unfinished features as completed work.

1.5 Research Questions (where applicable)

We asked how an SIS can retain an authoritative registration record while an external learning service is delayed or unavailable. We also asked how approval responsibilities, immutable evidence and clear interface states can reduce ambiguity in academic workflows. A further question was how to demonstrate a university configuration without hard-coding one institution's rules into every page.

The proposed AI question concerns whether governed assistance can combine permitted administrative and learning context without making academic or counselling decisions autonomously. We document the required controls, but we do not claim an evaluated AI implementation at this checkpoint.

1.6 Scope of the Project

Our implemented scope includes applicant and admissions journeys, formal registration, course planning, finance workflows, assessment and official-result controls, timetable authoring and publication, role-based workspaces, student self-service and a separate MoodleCloud provisioning demonstration. The local demonstration uses generated accounts and representative academic configuration.

Our wider scope includes advanced institutional configuration, attendance and examinations, proactive support, AI assistance and expanded reporting. Automatic registration-driven live Moodle delivery, live grade exchange and LTI launches remain incomplete. We excluded migration of live institutional student records, changes to Moodle core and training a language model from scratch. No real financial settlement or clinical counselling claim follows from the demonstration.

1.7 Significance of the Project

The practical contribution is a workflow-oriented prototype in which the same student identity connects administrative decisions, course registration and the student's view of

academic services. Staff can review records within assigned workspaces, and the student can see registration and timetable states without interpreting internal error codes.

The engineering contribution is the separation of authoritative decisions from external delivery. That separation allows the system to keep valid registration evidence when Moodle is unavailable. Our project also provides an inspectable source archive, repeatable demonstration preparation and explicit verification records that support supervision and future development.

1.8 Limitations

We had limited project time and no production institutional dataset. We therefore used generated participants and synthetic operating policies. These support workflow demonstrations but do not establish performance at university scale or the acceptability of every policy to the University.

External service configuration and deployment access affected integration work. The MoodleCloud demonstration proves the creation and accessibility of a bounded set of courses and memberships, rather than continuous synchronisation. The fresh source checkpoint also contains work newer than the last verified cloud release. We report these boundaries alongside the results. We did not conduct a completed controlled user study, clinical evaluation or AI accuracy experiment.

1.9 Organisation of the Report

Chapter 2 reviews technical foundations and related approaches. Chapter 3 explains our development and evaluation method. Chapter 4 presents requirements and design, including architecture, data relationships and interaction choices. Chapter 5 describes implementation and changes from the proposal. Chapter 6 reports verification evidence and evaluates the objectives. Chapter 7 discusses the implications and limitations. Chapter 8 concludes the report and recommends the next work. The appendices provide collaboration records, demonstration instructions, additional diagrams and supporting checks.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

We reviewed primary technical sources alongside the project proposal and controlling design handbook. The review focuses on interoperability, administrative authority, accessible interfaces and governed AI. It is a focused technical review, not a completed systematic review of the research literature. We exclude proposal entries whose titles, identifiers or links could not be substantiated.

Our comparison distinguishes what a standard enables from what our implementation demonstrates. A web service can create an enrolment, for example, but the institutional rule deciding whether that enrolment is legitimate still belongs to the SIS.

2.2 Key Concepts and Theories

We distinguish identity, authorisation and business authority. Identity establishes who signed in. Authorisation establishes which operations that identity can perform in a selected scope. Business authority establishes which system and responsible officer can make a particular academic decision. These distinctions inform our workspace and official-record designs [5].

A transaction groups related database changes. Idempotency allows a repeated command to return the existing outcome without creating a duplicate registration. An outbox records intended external work alongside the authoritative transaction. We use these concepts to separate registration completion from learning-platform delivery. Our framework treats an immutable snapshot as evidence of a decision at a point in time, rather than a mutable screen that silently changes after approval.

2.3 Existing Systems / Related Work

Moodle's web service administration supports custom services, selected functions, authorised accounts and tokens. Its documentation makes clear that function availability and account capabilities both matter [1]. This directly explains the configuration work in our demonstration: enabling REST alone did not make all provisioning functions available.

LTI 1.3 defines an interoperable way for a learning platform to launch an external tool with standard security messages [2]. It addresses a different interaction from creating courses through a REST API. We therefore treat REST provisioning and LTI tool launch as separate integration lanes. Our implementation has not yet completed the LTI lane.

For future AI assistance, NIST AI RMF provides a structure for governing, mapping, measuring and managing risk [3]. We use it to organise proposed controls and human oversight. This does not establish that an unimplemented assistant satisfies those controls.

2.4 Comparative Analysis

We compared three approaches: manual administration, one-off web service provisioning and a governed integration workflow. Manual administration can support a small demonstration but provides weak repeatability. One-off provisioning automates remote creation but does not itself reconcile later SIS changes. A governed workflow must retain local intent, scope, delivery status and recovery evidence.

LTI complements these approaches by providing a tool launch mechanism. It does not replace the institutional registration model. Table 2.1 compares the responsibilities and the resulting implementation choices. We make no comparative claim about commercial SIS products that we have not evaluated.

Table 2.1 Integration approach comparison

Approach	Responsibility	Project implication
Manual administration	Human creation and correction	Useful for setup, weak repeatability
REST provisioning	Remote users, courses and memberships	Demonstrated in a bounded cloud scenario
Governed delivery	Durable intent, approval and reconciliation	Live operational foundation incomplete
LTI 1.3	Standard tool launch context	Separate unimplemented lane

2.5 Research / Knowledge Gap

The gap we chose to address is the connection between an institutional decision and its visible operational consequences. The external API may work while the student's administrative state is inconsistent, or a page may report completion without a durable decision record. We designed the workflow around evidence of authority, validation, publication and recovery.

The remaining gap within our own implementation is equally specific: separate successful provisioning has not yet become a fully governed, repeatable live delivery process driven by formal registration. The report preserves that distinction so that the project contribution can be assessed without overstating integration completeness.

2.6 Conceptual or Theoretical Framework (where applicable)

Our conceptual framework follows a chain from requirement to authorised command, validated state change, recorded evidence and user-visible outcome. When the outcome requires an external system, a delivery record connects local intent to external execution and

reconciliation. This framework allows us to ask whether each workflow has a responsible actor, a valid transition and a recoverable failure state.

For the planned support layer, permitted evidence would enter a human-reviewed support workflow rather than directly changing grades, registration or financial status. WCAG 2.2 provides testable accessibility criteria for interface evaluation [4]. Automated scans form one part of that evaluation and do not establish complete conformance.

2.7 Chapter Summary

The review supports two design decisions: separate administrative authority from learning delivery, and evaluate integration lanes independently. It also supports explicit human oversight for proposed AI functions and a broader accessibility process than automated scans alone. These foundations guide the architecture and evaluation in the following chapters.

CHAPTER 3: METHODOLOGY

3.1 Introduction

We used an iterative software engineering approach governed by a design handbook and traceable task packets. We developed vertical slices that connect a user action to an API command, persisted records, audit evidence and a visible result. We evaluated each slice at the level its evidence could support.

3.2 Research / Development Approach

The proposal combined high-level planning with iterative development. In practice, we retained that combination while replacing its provisional technology stack. We first inspected the controlling design, supersession rules and approved evidence. We then used scoped task packets, source changes, tests and review notes to implement and refine each workflow [5].

We used isolated Git worktrees when concurrent development could affect another developer's files. A development slice was not considered equivalent to a complete release: connected checks, browser journeys and operational evidence remained separate gates. This approach helped us preserve working changes while resolving conflicting implementations.

3.3 Requirements / Data Collection

Our principal requirements inputs were the project proposal, the SIS–Moodle handbook, approved design evidence, blueprint journeys and implementation reviews. We also used direct demonstration feedback, including unclear document-upload states, unexpected workspace routing and timetable registration blockers. We translated these observations into reproducible checks rather than treating them as survey findings.

We prepared generated accounts and university-like academic data to exercise workflows without importing live student records. The preparation script uses a dedicated local review database and preserves existing matching registrations on repeat execution. We do not claim that the preparation dataset represents the demographic distribution of University of Zambia students.

3.4 System Architecture / Research Workflow

Figure 4.1 shows the application boundary. Next.js provides the web interface, NestJS provides a modular API and PostgreSQL persists transactional records through Prisma. Shared packages hold contracts, configuration and UI components. A workflow process performs supported background actions.

We connect the SIS to a simulator for repeatable integration scenarios and to separate live Moodle tooling for the bounded cloud demonstration. Formal registration commits locally

before external delivery. This is a modular monolith with distinct web and workflow processes, rather than the microservices architecture originally proposed.

3.5 Tools and Technologies

We implemented the application in TypeScript with Next.js, NestJS and PostgreSQL. Prisma provides schema migrations and data access. Playwright supports browser journeys, while unit and connected tests exercise service behaviour. GitHub hosts the repository and continuous integration. Vercel hosts the verified web and API release.

Docker Compose defines an isolated supervisor review environment. Our documentation pipeline produces editable reports, rendered PDFs, diagrams and a source archive. The original proposal's Django, MySQL, Celery, Redis and Qdrant stack does not describe the current implemented system. No deployed LLM gateway or vector database is claimed.

3.6 Ethical and Legal Considerations

We avoided live institutional student data in the demonstration and excluded external service tokens from the shareable source archive. We distinguish disposable local demonstration passwords from private Moodle credentials. Role and scope checks restrict access to records and operations, while sensitive decisions require the applicable review and audit evidence.

We acknowledge third-party documentation and software. Synthetic policies allow a presentation scenario to run without representing university approval. Future wellbeing and AI functions require a clear consent model, limited access, retention rules and human escalation. We have not presented an automated risk score as a diagnosis or an authorised academic decision.

3.7 Evaluation Method

We evaluated functions through unit checks, isolated PostgreSQL tests, normal API journeys and browser checks. We tested the visible response together with the persisted result where connected evidence was available. We also tested repeat preparation to detect destructive or duplicate behaviour.

For UI changes, we checked desktop and mobile layouts, keyboard interaction, independent scrolling and automated accessibility findings. We record test counts with their checkpoint and scope, rather than combining unrelated runs into a single completion percentage. We did not measure a statistically supported reduction in administrative workload, large-scale throughput or student satisfaction.

3.8 Chapter Summary

Our method links handbook requirements to executable journeys and recorded evidence. We separate local source checks, earlier release verification and human observations.

CHAPTER 4: SYSTEM ANALYSIS AND DESIGN

4.1 Introduction

We designed the SIS around the decisions that university users need to make. Requirements cover the actors, their authorised scope, the records they may change and the evidence needed to explain the outcome. We use diagrams to show boundaries and relationships rather than treating interface pages as independent features.

4.2 Stakeholder and User Analysis

Applicants need programme discovery, clear requirements, resumable applications and visible document status. Students need registration, course and finance information, results, timetables and support without staff-only navigation. Lecturers need assigned academic workflows and the ability to distinguish provisional work from official decisions.

Registry, admissions, finance, timetable and Moodle officers require different workspaces and scopes. Reviewers need sufficient evidence to approve independently. Administrators configure access and institutional structure, but their role does not make every academic decision valid. Table 4.1 summarises these actors and their primary boundaries.

Table 4.1 Principal actors and authority

Actor	Primary work	Boundary
Applicant	Application and evidence	Own application
Student	Registration and academic services	Own records and published views
Lecturer	Assigned assessment work	Assigned offerings and permissions
Registry and reviewers	Academic decisions	Assigned scope and independent review
Timetable staff	Draft and publish sessions	Validated resources and review
Moodle administrator	Mappings and integration	Authorised integration scope

4.3 Functional Requirements

The system must support applications and uploaded evidence, formal registration with policy and course checks, scoped academic administration, finance records and assessment workflows. It must support timetable validation and publication with a student-specific view. It must also expose the state of Moodle work instead of equating a queued action with successful delivery.

Across these functions, the system must persist actor, scope and decision evidence and reject unauthorised or invalid transitions. The student portal must communicate what needs attention and provide a route to the relevant action. Table 4.2 records representative requirements and their verification basis.

Table 4.2 Representative requirements

Requirement	Visible outcome	Evidence basis
Formal registration	Receipt and recorded course context	Connected local journeys
Scoped workspace	Valid direct landing	Focused authentication checks
Timetable publication	Student-specific published sessions	Publication and UI checks
Moodle provisioning	Visible learner courses	Separate cloud demonstration
Recoverable integration	Explicit unsupported state	Source boundary and focused checks

4.4 Non-Functional Requirements

Our non-functional requirements address confidentiality, integrity, recoverability, usability, accessibility and maintainability. We require server-side checks for authority and scope, predictable handling of repeated commands and preservation of official decision evidence. We require a clear error when an external capability is unavailable.

The interface must remain usable with long navigation lists and large record sets. Shared components support status labels, loading feedback, filtering and paginated views where implemented. We avoid claiming university-scale capacity from the small demonstration dataset. Performance and resilience require separate measured evaluation before institutional operation.

4.5 Use Cases / User Stories

A representative registration use case starts when a student selects a period and prepares a course plan. The system validates course and policy conditions, presents declarations and records a successful formal registration only when the required checks pass. A repeated submission must preserve the same successful outcome.

A timetable use case starts with a coordinator preparing a master draft. Validation checks the relevant resources and clashes. An independent reviewer decides whether the proposed publication can proceed. The student receives only sessions belonging to the applicable

published offering and registration context. A Moodle use case begins with authorised local intent and must distinguish queued, attempted, failed and confirmed external work.

4.6 System / Solution Architecture

The web application presents role-specific interfaces and calls the API. The API coordinates domain services and persists records in PostgreSQL. Shared contracts reduce disagreement about command and response shapes. An outbox separates durable intent from external processing. Figure 4.1 illustrates these boundaries.

Figure 4.2 shows the registration sequence. The database transaction records registration evidence and queued integration intent together. A later Moodle failure does not remove the valid institutional registration. The current live delivery foundation stops before unsupported external actions, leaving the intended work visible for recovery.

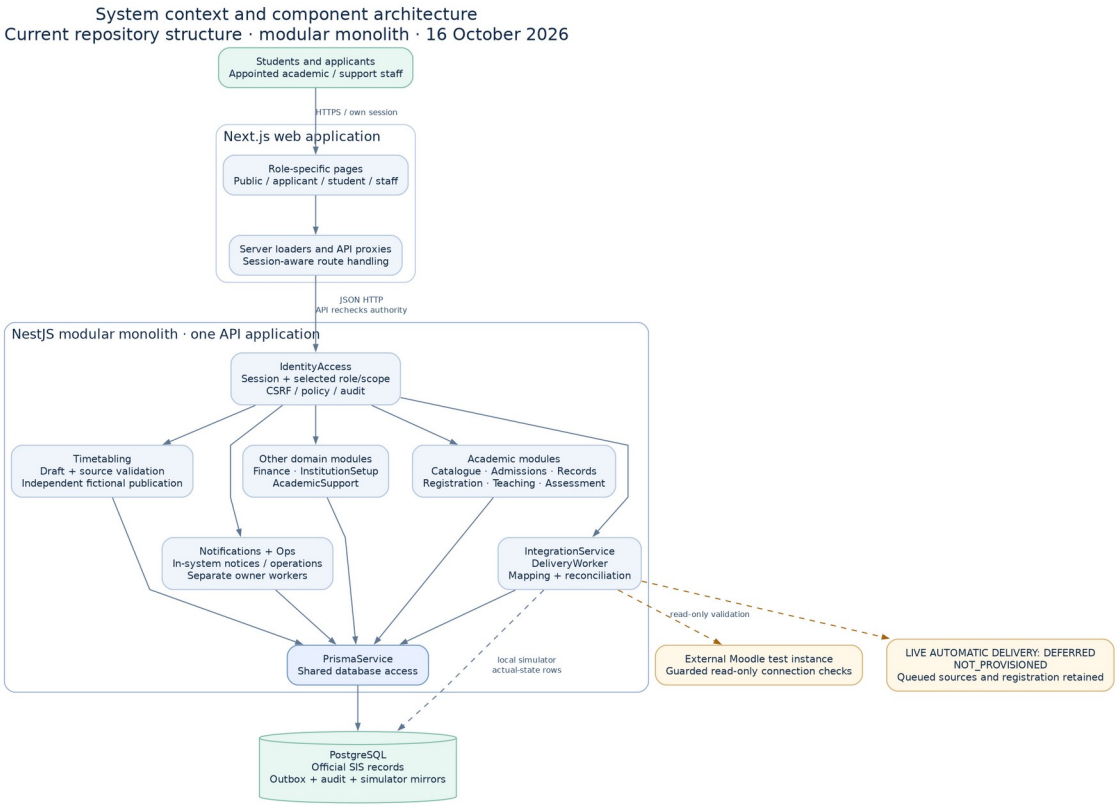


Figure 4.1 System context and component boundaries

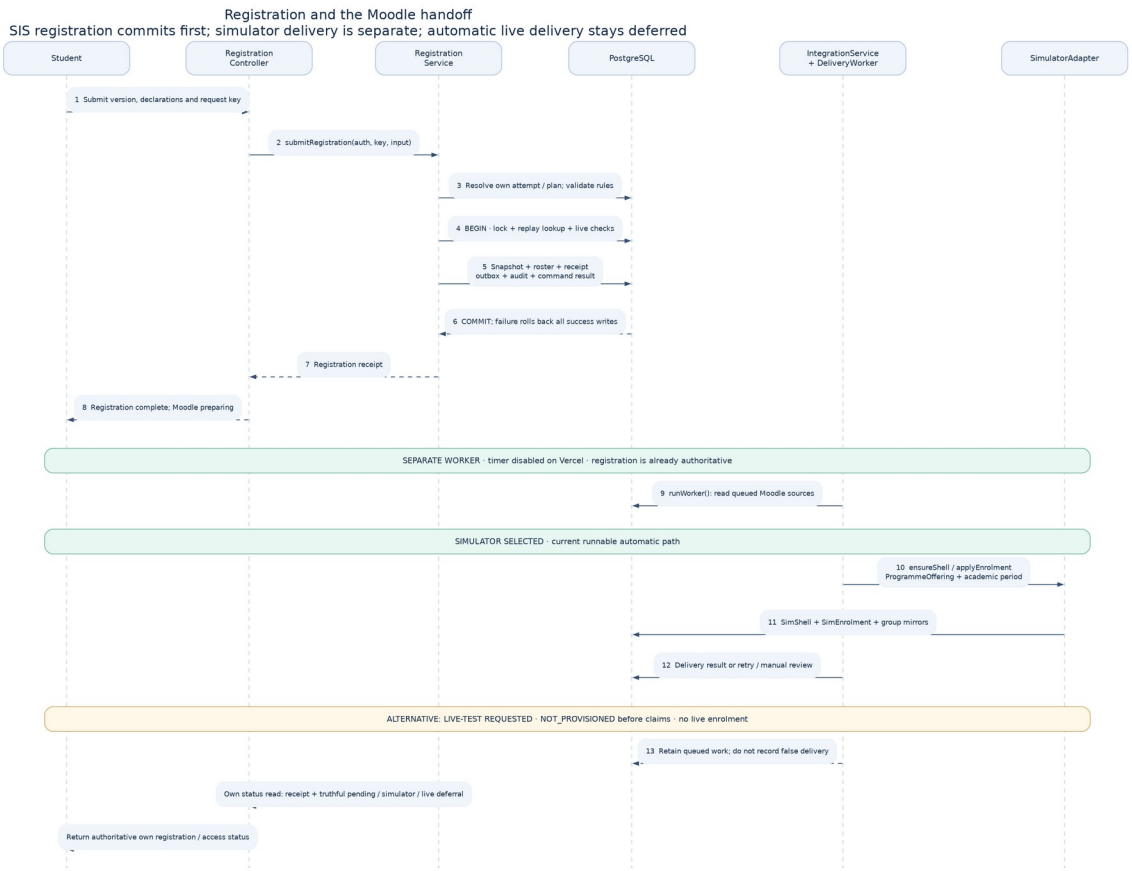


Figure 4.2 Registration and deferred Moodle delivery sequence

4.7 Database / Data Design (where applicable)

Figure 4.3 presents selected domain relationships. Institutional registration records the formal decision for a period and retains a snapshot and receipt. Course registrations connect the student to the relevant course context. Outbox and command records support repeatability and delivery evidence. Timetable publication records retain the approved set of sessions and the student's projection.

We distinguish relational constraints from scalar identifiers that do not establish a database foreign key. We also distinguish a programme offering used for a Moodle shell key from individual course delivery contexts. These distinctions matter when implementing mapping, reconciliation and audit queries. The diagram is a selected model, rather than an assertion that it contains every table.

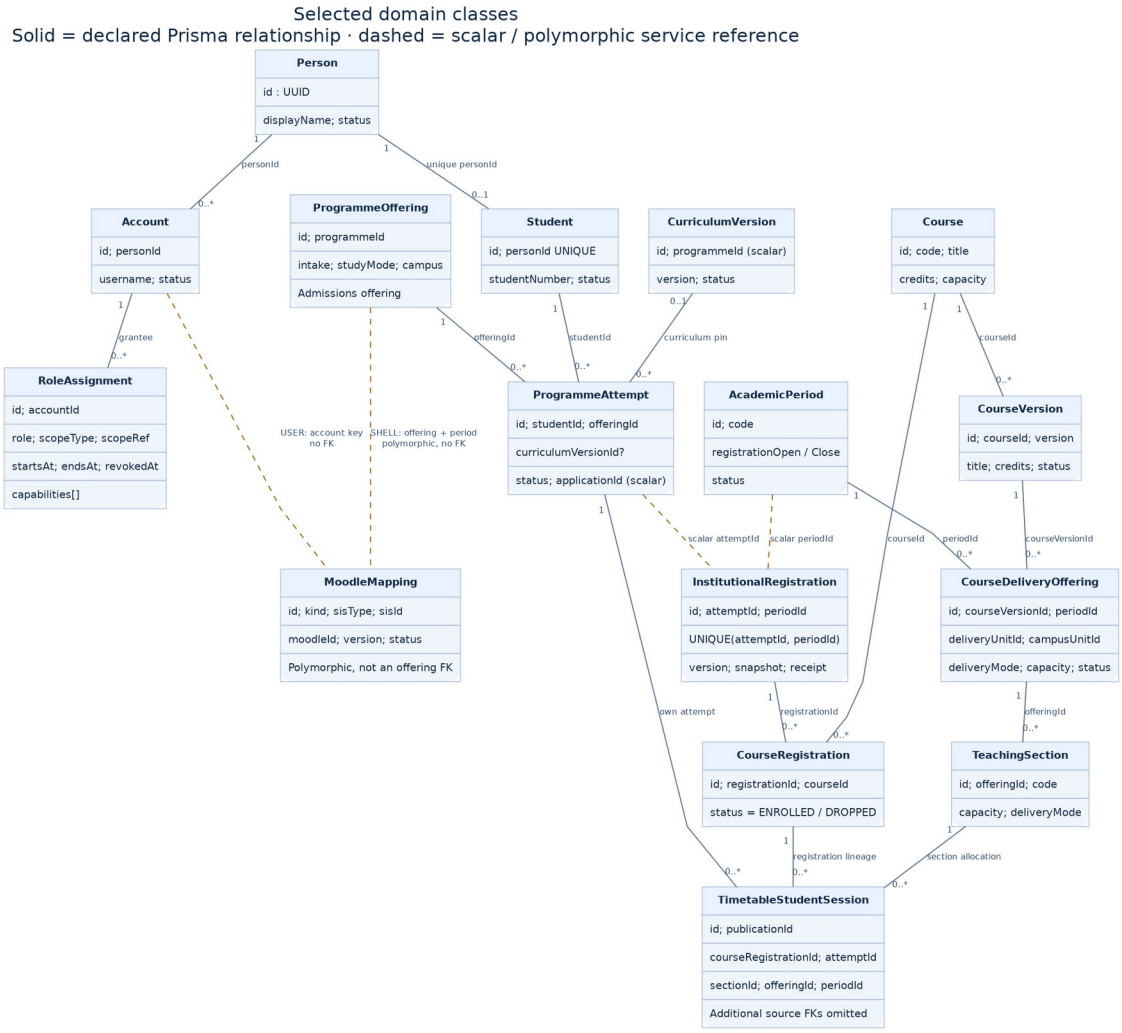


Figure 4.3 Selected domain class relationships

4.8 Interface / Interaction Design (where applicable)

We designed workspaces around stable navigation and layered information. The staff interface uses a dark green sidebar, a workspace header and structured cards for forms, queues and tables. The student redesign applies the same navigation discipline with a persistent account area, separate navigation scrolling and a main content region.

The student home page prioritises academic status and useful actions. Timetable cards expose publication state and class details rather than presenting an unstructured vertical text list. Mobile layouts collapse navigation into an accessible control. Figures 4.4 and 4.5 show the connected student home and a desktop timetable view with controlled demonstration responses. We preserve empty, loading, warning and unavailable states rather than displaying invented totals.

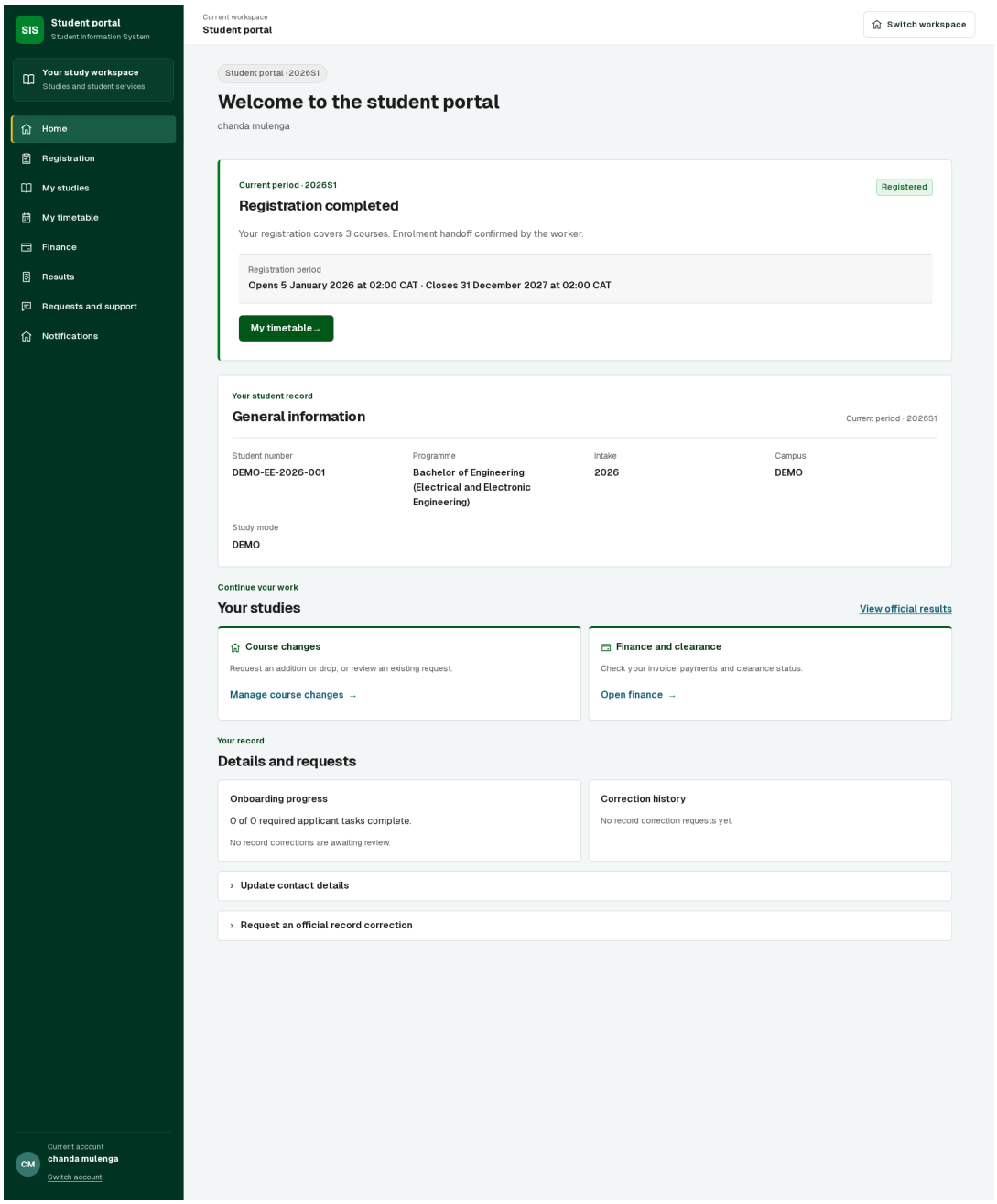


Figure 4.4 Connected student home with simulator enrolment handoff

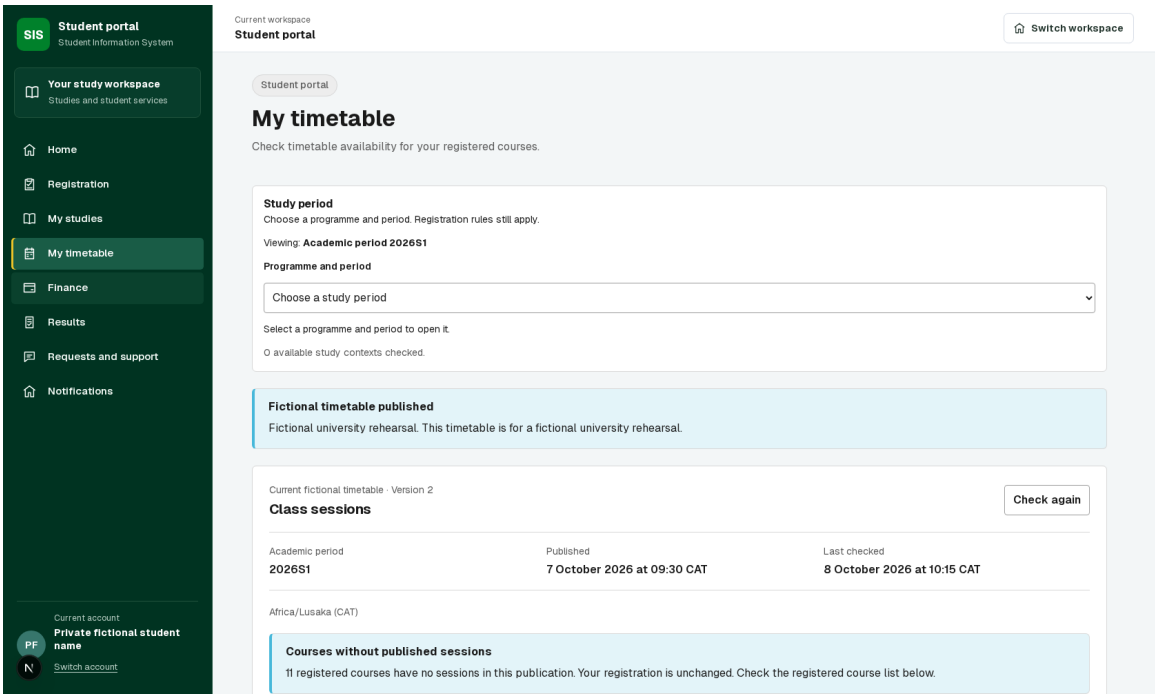


Figure 4.5 Student timetable view with controlled demonstration responses

4.9 Algorithms / Models / Technical Design

Our key technical decisions concern validation order, idempotency and publication. A formal registration command validates its input and authority, checks the current plan and policy, and commits related records together. Repeated commands use command evidence to avoid duplicate successful registrations.

A timetable draft is not published merely because its sessions exist. The publication workflow checks source validity and conflicts, applies independent review and creates a versioned published result. Figure 4.6 shows that activity. Figure 4.7 describes selected service boundaries. These designs keep preparation, decision and delivery states distinct, which makes failures easier to explain and recover.

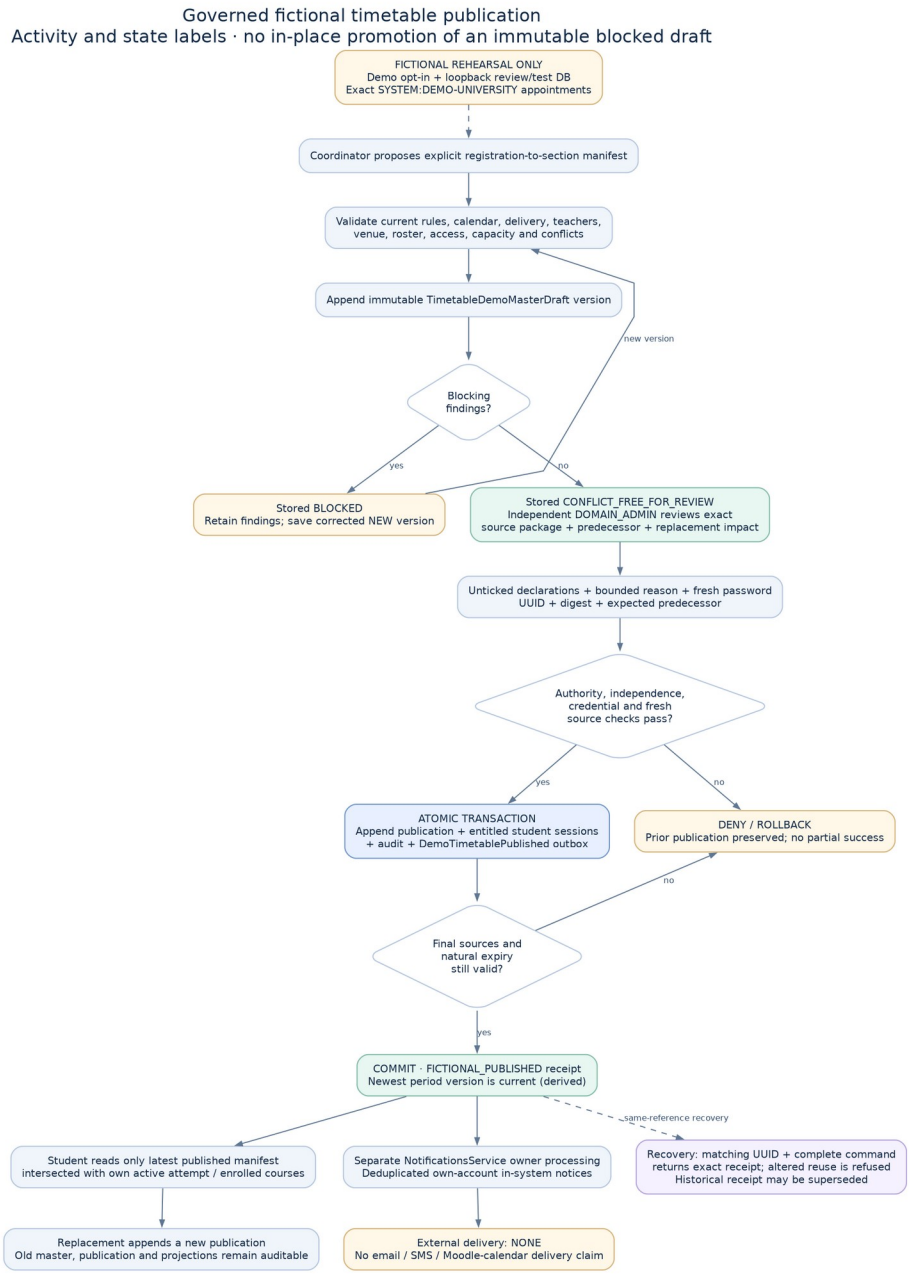


Figure 4.6 Timetable publication activity

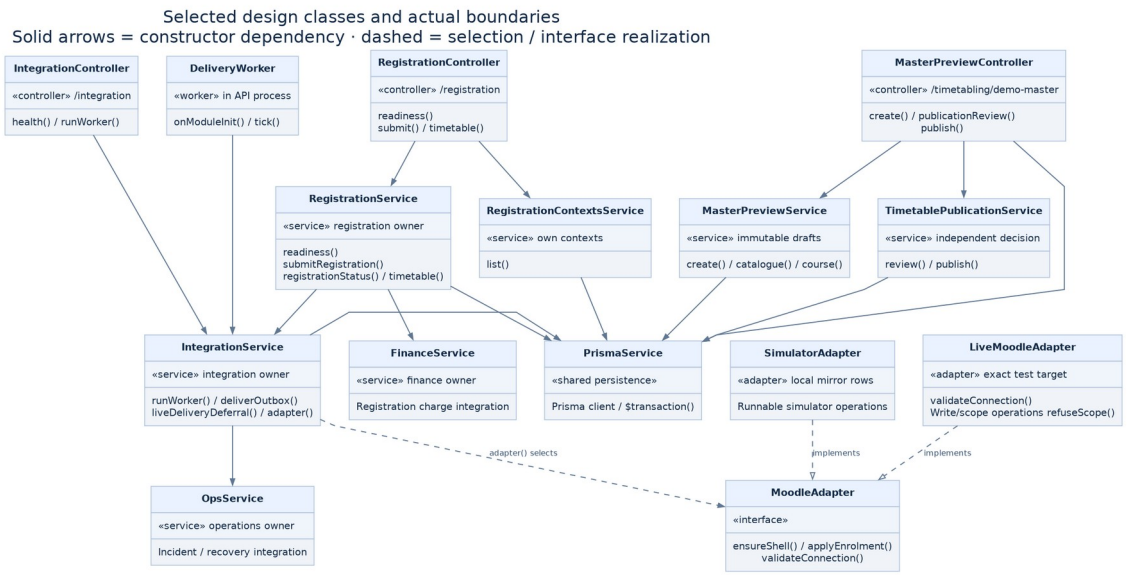


Figure 4.7 Selected design class boundaries

4.10 Chapter Summary

The design connects a user-facing action to a defined actor, command, state transition and evidence record. Shared interfaces make the system coherent, while domain-specific checks preserve the different responsibilities of admissions, registration, assessment and timetable publication.

CHAPTER 5: IMPLEMENTATION / DEVELOPMENT

5.1 Introduction

We implemented a working SIS foundation and developed successive workflow slices on top of it. This chapter describes the current source checkpoint and the separate cloud demonstration. It distinguishes changes that are in the verified deployed release from newer presentation and authentication work.

5.2 Development Environment

Our development workspace contains the application under development/ and the controlling handbook beside it. The application uses npm workspaces and shared packages. The portable review configuration fixes the Node runtime and provides an isolated PostgreSQL database. Database migrations and generated data preparation establish a repeatable starting point.

We provide a source archive with a file manifest and checksums. The archive excludes local environment secrets, external credentials, dependency folders and private evidence. The local startup guide identifies expected ports and demonstration accounts. The clean container build remains a separate verification item because the registry download timed out during the recorded run.

5.3 System Components / Modules

The implemented application includes applicant discovery and evidence submission, admissions handling, registration and course planning, finance administration, assessment plans and grade workflow controls, timetable preparation and publication, identity and scoped workspaces. Shared UI elements provide page headings, cards, tables, actions, status communication and form feedback.

The current student presentation changes add sidebar navigation, a pinned account area, improved home summaries and timetable cards. Authentication changes route the user to a valid selected workspace rather than forcing a workspace-choice page on every sign-in. The explicit switcher remains available when a user needs a different authorised context.

5.4 Key Implementation Details

We treated optimistic version values, prerequisites and capacity findings as business-state problems rather than hiding their messages. Registration screens need to show which course or condition blocks submission and route the student back to the appropriate correction. We retained checks that prevent invalid plans from becoming formal registrations.

For authentication, a selected workspace can be retained only when it remains valid for that account. Invalid, revoked or foreign context does not become authority through a remembered browser choice. For local demonstration preparation, a strict database guard

prevents the review script from accidentally populating a different database. Repeated preparation preserves matching registrations and timetable history.

5.5 Integration

Our separate MoodleCloud demonstration uses the existing site, a configured external service and a token stored outside the source archive. The provisioning tooling created four generated accounts, twelve visible course shells and forty-eight memberships. The user confirmed that a learner could sign in normally, see the courses and open a course [6].

That result proves the bounded provisioning and learner-access scenario. It does not prove registration-driven live synchronisation, course-change reconciliation, automatic grade exchange or LTI launch. The normal SIS demonstration still uses the integration simulator. Unsupported live operations return an explicit not-provisioned outcome rather than reporting a fabricated success. Figure 4.2 shows the intended boundary between durable local intent and later delivery.

5.6 Challenges and Solutions

We encountered concurrent changes, inconsistent demonstration state, unclear error messages and interface layouts that did not support long workspaces. Isolated worktrees and explicit file review reduced interference. Source and runtime checks helped distinguish a genuinely missing capability from a stale deployed page or incomplete configuration.

The Moodle configuration work showed that REST, service membership, function exposure and account permissions must agree. The student UI work showed that making two pages attractive did not make the whole workspace coherent. We addressed navigation through shared layout changes and preserved outstanding page-level work in the completion sequence.

5.7 Individual Contribution (group projects)

We, Chitundu Milimbo and Charles Hangoma, worked as partners on the project. Our proposal allocated an integration emphasis to Charles and an AI/LLM emphasis to Chitundu. That planned division does not prove exclusive authorship of the current code or completion of the AI layer.

We record the shared implementation and review process in Appendix A and provide a task exchange packet. Each of us must confirm the tasks we personally completed using commits, task packets and review evidence before making an individual academic authorship declaration. We do not invent contribution percentages or attribute all assistant-supported changes to one partner.

5.8 Chapter Summary

The implementation demonstrates connected administrative workflows, a repeatable local dataset and separate live Moodle provisioning. The major unfinished integration work is the transition from one-off provisioning to a durable, authorised live delivery and reconciliation process. The AI and LTI ambitions remain visible in the project scope and objective evaluation.

CHAPTER 6: TESTING, RESULTS AND EVALUATION

6.1 Introduction

Our evaluation asks whether the observed behaviour matches the requirement and whether the evidence supports the claim. We separate earlier release checks from current local changes. A successful build or a configured external service alone is insufficient evidence of a completed user journey.

6.2 Testing Strategy

Unit tests check focused logic. Connected PostgreSQL tests check persisted behaviour and transaction outcomes. Browser tests exercise navigation and user actions through the interface. Automated accessibility scans identify a bounded set of machine-detectable issues. Human checks complement automation for external learner access.

We use positive and negative scenarios. Valid registration must create the intended evidence. Invalid authority, stale context and unsupported live integration must fail clearly. Repeat preparation must avoid duplicate or destructive changes. We do not add together the different test runs as if they formed a single comprehensive release suite.

6.3 Test Cases and Results

Table 6.1 records representative checks and the evidence available at this checkpoint. The earlier merged release passed its recorded continuous-integration run. Newer student and authentication work has focused local evidence and still needs a combined release run before being represented as a new deployed baseline.

An initial broad API run against an unseeded database failed IAM expectations, and concurrent package rebuilding caused additional import failures. After stabilising the build, the ten affected suites passed 267 checks. That recovery does not erase the need for a fresh, correctly prepared full checkpoint run. We therefore report the failure and the narrower recovery result separately.

Table 6.1 Verification evidence by checkpoint

Check	Recorded outcome	Interpretation
Earlier release CI	137 browser, 26 Moodle deferral and 490 API unit checks passed; 13 existing skips	Earlier merged release only
Connected authentication	6 passed	Current focused local checks
Authentication browser	4 passed	Current routing journeys

Check	Recorded outcome	Interpretation
Generated participant logins	4 passed	Local review dataset
Student UI	39 checks and 5 automated accessibility scans passed	Controlled scenarios, not full accessibility certification
Import recovery run	267 checks passed in 10 suites	Narrow recovery after broad-run failures
Clean Compose startup	Not completed	Registry timeout, configuration validated
Moodle learner access	User confirmed courses appeared and opened	Cloud provisioning demonstration

6.4 Experimental Results / Performance Results (where applicable)

The local supervisor dataset contains nine formal registrations and thirty-six associated course registrations. It also contains a twelve-session master timetable draft for review. Four generated participant logins passed connected browser checks. Repeating the workflow preparation preserved existing matching registrations and timetable history [7].

The separate MoodleCloud dataset contains twelve courses and forty-eight memberships across four generated accounts. These counts describe the demonstration's size. They are not throughput, latency or scaling measurements. We did not establish an 80% coverage result, a maximum concurrent-user capacity or a measured improvement in administrative processing time.

6.5 User Evaluation (where applicable)

The recorded human observation is the learner-access confirmation for MoodleCloud: the courses appeared and a course opened after normal sign-in. We also used development feedback to identify navigation and registration usability problems.

We have not completed a structured evaluation with a recruited sample of university students or staff. We therefore report no satisfaction percentage, task-completion average or institutional acceptance decision. Appendix E provides a proposed observation instrument for a later supervised evaluation, clearly separated from collected findings.

6.6 Analysis of Results

The strongest result is that authoritative SIS workflows can be exercised with persisted evidence and visible student outcomes. The verification also supports the current sign-in routing and generated participant access within the tested scenarios. Timetable and student navigation work has specific UI evidence.

The main integration limitation remains operational continuity. A successful provisioning script proves remote writes, but a complete integrated SIS must also process later registration changes, recover partial failures and reconcile differences. The current evidence supports the former and identifies the latter as unfinished. We make the same distinction between a planned governed AI workflow and an evaluated AI service.

6.7 Objective-by-Objective Evaluation

Table 6.2 evaluates the two original proposal objectives. We achieved substantial core SIS implementation and a bounded Moodle provisioning demonstration. We have not achieved the complete bidirectional and LTI integration objective. The governed AI objective remains unfulfilled as an implemented and evaluated capability.

We retain the original objectives because removing their unfinished portions would conceal an important project result. The current implementation offers a useful foundation and presentation demonstration, while the remaining work is necessary to meet the proposal in full.

Table 6.2 Original objective evaluation

Original objective	Achieved evidence	Remaining work
1 Core SIS and Moodle integration	Core workflows; separate 12-course cloud provisioning	Automatic live delivery, grade exchange and LTI
2 Governed AI and proactive support	Governance design and workflow foundations	Implemented assistant, permitted retrieval, measured safety and human evaluation

6.8 Individual Results (group projects)

The test records currently describe shared project changes rather than an audited division of results between the two partners. Our individual result statements must point to the specific tasks and checks each of us performed. Appendix A provides the names, planned areas of responsibility and the review exchange procedure without claiming unsupported personal test totals.

6.9 Chapter Summary

The available evidence supports a functioning SIS prototype, specific student-interface improvements and separate live Moodle provisioning. It does not support a claim that every proposed feature is complete, that the whole new checkpoint has passed release verification or that the system is ready for institutional production.

CHAPTER 7: DISCUSSION

7.1 Introduction

We discuss what the implemented workflows establish and where the evidence remains limited. The central issue is whether the system connects legitimate institutional decisions to reliable operational outcomes, including during external failure.

7.2 Interpretation of Findings

Our results indicate that administrative authority can remain local while integration work is recorded separately. This matters because external downtime should not silently invalidate an already completed registration. It also means the interface must explain both states clearly so a student does not interpret delayed Moodle access as failed registration.

The UI changes show the value of a shared workspace structure. Persistent navigation and a visible account area support orientation across multiple pages. However, a common shell cannot substitute for coherent forms, useful tables and understandable errors on every individual page. Further interface work should follow complete journeys rather than isolated screenshots.

7.3 Comparison with Existing Work

Moodle web services provide the remote capabilities used by our provisioning demonstration [1]. Our additional work concerns the local authority, state and evidence surrounding those calls. LTI offers a separate standard for launching tools [2], which remains a valid future lane rather than an alternative name for the completed REST demonstration.

NIST's risk framework informs the proposed governance of the AI layer [3]. We have not evaluated an AI model or shown that AI improves student outcomes. Accessibility criteria similarly guide evaluation [4], while our automated scans cover only part of the required human experience. We avoid presenting standards alignment as completed certification.

7.4 Contributions of the Project

Our practical contribution is a connected, inspectable academic administration prototype with representative student and staff workflows. Our technical contribution includes explicit state transitions, idempotent commands, scoped workspaces, versioned evidence and a separation between authoritative records and external delivery.

We also provide a reusable local demonstration setup, generated participants, source packaging and engineering diagrams. These support supervision and reproducibility. The project does not claim a new machine-learning algorithm, a completed university deployment or measured superiority over an existing institutional system.

7.5 Limitations and Implications

The small generated dataset limits conclusions about scale and institutional diversity. Synthetic policies demonstrate configurability but require responsible institutional review before real operation. A user-confirmed Moodle course view is useful evidence, but it does not exercise all later synchronisation and recovery cases.

The proposal was broader than the implemented and evaluated checkpoint. LTI, automatic live delivery, governed AI, advanced support and several extended institutional workflows remain incomplete. The implication is that our presentation should demonstrate the functioning core accurately while showing a concrete continuation path, rather than describing the project as a finished enterprise product.

7.6 Chapter Summary

The project supports its core design principle: a reliable SIS must make authority, workflow state and external delivery explicit. The remaining work concerns completion and evidence across the whole lifecycle, particularly live integration, support governance and institutional evaluation.

CHAPTER 8: CONCLUSION AND RECOMMENDATIONS

8.1 Introduction

We conclude by relating the demonstrated implementation to the original aim and by identifying the work needed to complete the wider proposal. Our recommendations follow the evidence reported in Chapter 6.

8.2 Summary of the Project

We designed and implemented a modular SIS that links applicant, registration, academic, finance and timetable workflows through scoped interfaces and persisted decisions. We improved student navigation and prepared a repeatable local demonstration dataset. We also provisioned a separate MoodleCloud course environment and obtained a learner-access confirmation.

We used the controlling handbook and task packets to guide implementation, then evaluated specific workflows through tests and observations. The implemented stack differs from the original proposal but retains its aim of authoritative administration with learning-platform integration.

8.3 Conclusions

We conclude that the project provides a working foundation for SIS–Moodle integration and a demonstrable set of administrative workflows. The separation between registration authority and external delivery is supported by the architecture and the tested local journeys.

We cannot conclude that full bidirectional integration, LTI tools or governed AI assistance is complete. We also cannot claim institution-wide usability, large-scale capacity or production readiness from the current evidence. These conclusions define the project's actual contribution and the remaining obligations.

8.4 Achievement of Objectives

Objective 1 is partially achieved through core SIS workflows and the separate Moodle provisioning demonstration. Automatic live synchronisation, grade exchange and LTI embedding still require implementation and connected verification. Objective 2 has design and governance foundations but lacks the implemented and evaluated AI functions needed for completion.

Table 6.2 provides the detailed assessment. We retain this evaluation in the final report so that an examiner can distinguish demonstrated capability from the broader planned system.

8.5 Recommendations

We recommend completing the durable live Moodle delivery foundation first, including scope validation, independent approval where required, dry-run evidence, partial-failure recovery and reconciliation. We then recommend running a fresh combined verification of the current student and authentication changes before merging and deploying them.

We recommend a supervised user evaluation using complete student and staff tasks. Real operation would additionally require approved institutional configuration, data governance, measured performance, backup and restore exercises and responsible ownership of support workflows. Private service credentials should remain outside the shareable academic submission.

8.6 Future Work

Future implementation should complete live registration changes and grade ingestion, then evaluate LTI launches as a separate lane. Attendance, examinations, progression, graduation, reporting and advanced admissions configuration need complete journeys and evidence. Protected student support and AI assistance should follow explicit consent, retrieval scope and human review rules.

We also plan to extend consistent information cards, filtered tables, upload progress, loading states and responsive navigation across remaining pages. A complete product requires these improvements together with operational verification, rather than a set of attractive pages without connected behaviour.

REFERENCES

[1] Moodle, “Using web services,” MoodleDocs. https://docs.moodle.org/503/en/Using_web_services. Accessed 16 October 2026.

[2] IMS Global Learning Consortium, “Learning Tools Interoperability Core Specification 1.3,” final release, 2019. https://www.imsglobal.org/spec/lti/v1p3/. Accessed 16 October 2026.

[3] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework,” 2023. https://www.nist.gov/itl/ai-risk-management-framework. Accessed 16 October 2026.

[4] World Wide Web Consortium, “Web Content Accessibility Guidelines 2.2,” W3C Recommendation. https://www.w3.org/TR/WCAG22/. Accessed 16 October 2026.

[5] C. Milimbo and C. Hangoma, SIS–Moodle design handbook, approved design evidence and implementation task packets, project repository, 2026. Internal engineering sources.

[6] C. Milimbo and C. Hangoma, “UNZA Catalogue and Moodle Demonstration,” development/docs/demo/UNZA-CATALOGUE-AND-MOODLE-DEMONSTRATION.md, project repository, 2026.

[7] C. Milimbo and C. Hangoma, “Supervisor review verification record,” development/docs/supervisor/VERIFICATION.json and associated local test evidence, 16 October 2026.

[8] C. Milimbo and C. Hangoma, “Student Information Management System Design and Implementation with Integration of Moodle,” original project proposal, 2026. Unpublished project document.

APPENDICES

APPENDIX A: INDIVIDUAL CONTRIBUTION STATEMENT (GROUP PROJECTS)

Project: Student Information Management System Design and Implementation with Moodle Integration

Partners: Chitundu Milimbo (2020992574) and Charles Hangoma (2021414469). Supervisor: Professor Jackson Phiri. Group size: two.

Our proposal identified Charles as the software engineering/integration specialist and Chitundu as the software engineering/AI specialist [8]. These are planned areas of emphasis, not an audited allocation of completed source code. We have not assigned unsupported contribution percentages.

Our task exchange packet: the implementing partner identifies the requirement, changes, test data and recovery behaviour. The reviewing partner checks authority, database effects, student-facing wording and failure scenarios. We exchange roles for the next packet. We record the review result and unresolved findings before integration.

For each individual statement, we will identify the task packet, files or commits, design decisions, tests personally performed and the other partner's contributions. We will distinguish shared implementation from independent review and acknowledge assistance used.

Chitundu Milimbo signature: _____ Date: 16 October 2026

Charles Hangoma signature: _____ Date: 16 October 2026

Supervisor: Professor Jackson Phiri

Supervisor verification signature: _____ Date: 16 October 2026

APPENDIX B: USER MANUAL / INSTALLATION GUIDE

We extract the supervisor source archive into a local folder and read READ-ME-FIRST.md. With Docker and Docker Compose available, we run: docker compose -f compose.supervisor.yml up --build. This downloads dependencies and images on a first build. We then open http://localhost:8088. The API uses localhost:8089. The dedicated review volume keeps demonstration data separate from other databases.

We sign in as sis.demo.chanda.mulenga with Review-2026-Chanda! for the student portal. Additional students are sis.demo.bwalya.mwansa with Review-2026-Bwalya! and sis.demo.mutale.banda with Review-2026-Mutale!. The lecturer account is

sis.demo.nasilele.tembo with Review-2026-Nasilele!. These are disposable local demonstration credentials, not the MoodleCloud passwords.

We check the home cards, registration periods, course registrations and timetable publication state. The lecturer can inspect academic assessment work. Timetable staff prepare and independently review publication using the documented seeded workspaces. A draft does not appear as a published student timetable. We preserve the local volume unless we deliberately intend to remove demonstration records.

The Compose configuration and native preparation have evidence, but the recorded clean container build was interrupted by a registry timeout. A supervisor should expect a first-build verification step. We keep private Moodle tokens outside the archive and use the separate cloud setup instructions when testing the live demonstration.

APPENDIX C: ADDITIONAL SYSTEM DESIGNS / DIAGRAMS

The accompanying diagram atlas contains the system context and components, registration-to-Moodle sequence, selected domain class model, design class boundaries and timetable publication activity. Editable Mermaid descriptions and companion render sources accompany the diagrams. Figures 4.1–4.7 include the principal views and current student interface evidence.

Our diagram boundaries distinguish implemented local transactions from deferred live delivery. The domain diagram is selected and does not claim to document every schema field. We use the vector atlas for readable inspection of larger relationships rather than relying on a reduced page image alone.

APPENDIX D: ADDITIONAL TEST CASES AND RESULTS

We retain logs and machine-readable results for the connected authentication tests, participant login browser checks, student UI checks, migration preparation and repeated local workflow preparation. The release record identifies its own revision and CI run. These records should be read with Table 6.1 because different checks cover different checkpoints.

We include negative scenarios for invalid workspace context, unauthorised access and unavailable live integration. We preserve observed broad-run failures instead of replacing them with the later narrower recovery result. Before the next release, we need a fresh complete run against a correctly migrated and seeded isolated database.

APPENDIX E: DATA COLLECTION INSTRUMENTS

Our proposed supervised observation form records participant role, task, starting state, completion outcome, blocking message, assistance required and the participant’s comments. Suggested tasks include finding a programme, uploading a qualification document,

completing a valid registration, locating a published timetable and inspecting an assessment decision.

We would obtain consent, avoid collecting unnecessary personal data and separate observation notes from authoritative student records. We would record the actual sample and method before reporting aggregate results. This instrument has not yet produced a completed participant dataset, and we report no invented survey findings.

APPENDIX F: ADDITIONAL CODE / CONFIGURATION

Our source archive includes compose.supervisor.yml, development/Dockerfile.supervisor and preparation scripts. The application source lives under development/apps and shared packages under development/packages. Database migrations and seed scripts define the review starting state.

The key registration transaction records the formal decision, snapshot, receipt, audit evidence, command outcome and outbox intent together. External work follows that committed intent. We reference the full source archive rather than copying large code listings into this report. Environment secrets and private cloud credentials are intentionally excluded.

APPENDIX G: OTHER SUPPORTING MATERIAL

We provide the editable defence presentation, this Word report, the rendered PDF, a diagram atlas and a runnable source checkpoint for supervision. The original report and presentation templates remain unchanged. We use 16 October 2026 as the preparation date.

Our release handover must still distinguish the previously verified cloud deployment from the latest local source changes. We will record the exact merged revision, deployment result and user checks when the new checkpoint completes release verification.

FINAL REPORT SUBMISSION CHECKLIST

Completed: project title, partners, computer numbers, supervisor and preparation date.

Completed: eight-chapter account, objective evaluation, diagrams, references and demonstration instructions.

Pending personal action: each partner verifies their contribution and signs the declaration.

Pending supervisor action: Professor Jackson Phiri reviews and signs the approval page.

Pending engineering work: combined release verification, automatic live Moodle delivery, LTI and evaluated AI functions.